

RECOMENDACIÓN GENERATIVA · TOKENIZACIÓN CONTEXTUAL

ActionPiece

Contextually Tokenizing Action Sequences for
Generative Recommendation

Seminario · Sistemas Recomendadores

Yupeng Hou · Jianmo Ni · Zhankui He · Noveen Sachdeva · Wang-Cheng Kang · Ed H. Chi · Julian McAuley · Derek Zhiyuan Cheng

Presentado por: **Benjamín Suazo y Pablo Lagos**

09 de junio de 2026



- 1 Contexto**
Recomendación generativa
- 2 El problema**
Tokenización sin contexto
- 3 Contribución y Marco teórico**
Tokenizador de acciones consciente del contexto
- 4 Solución**
Vocabulario ponderado y SPR
- 5 Resultados**
Evaluación y análisis
- 6 Conclusión**
Ideas clave

¿Qué es la Recomendación Generativa (GR)?

- **Recomendación secuencial:** predecir el próximo ítem dado el historial (SASRec).
- **Recomendación generativa (GR):** Convierte las acciones del usuario (sus interacciones) en patrones de tokens (identificadores) para predecir la siguiente acción de forma autoregresiva.
- **El problema central:** Los sistemas tokenizan las acciones/ítems de manera **independiente** al contexto.
 - ➔ Asignan el mismo "token" a un ítem sin importar qué compró el usuario antes o después.

• **Reconoce el contexto**

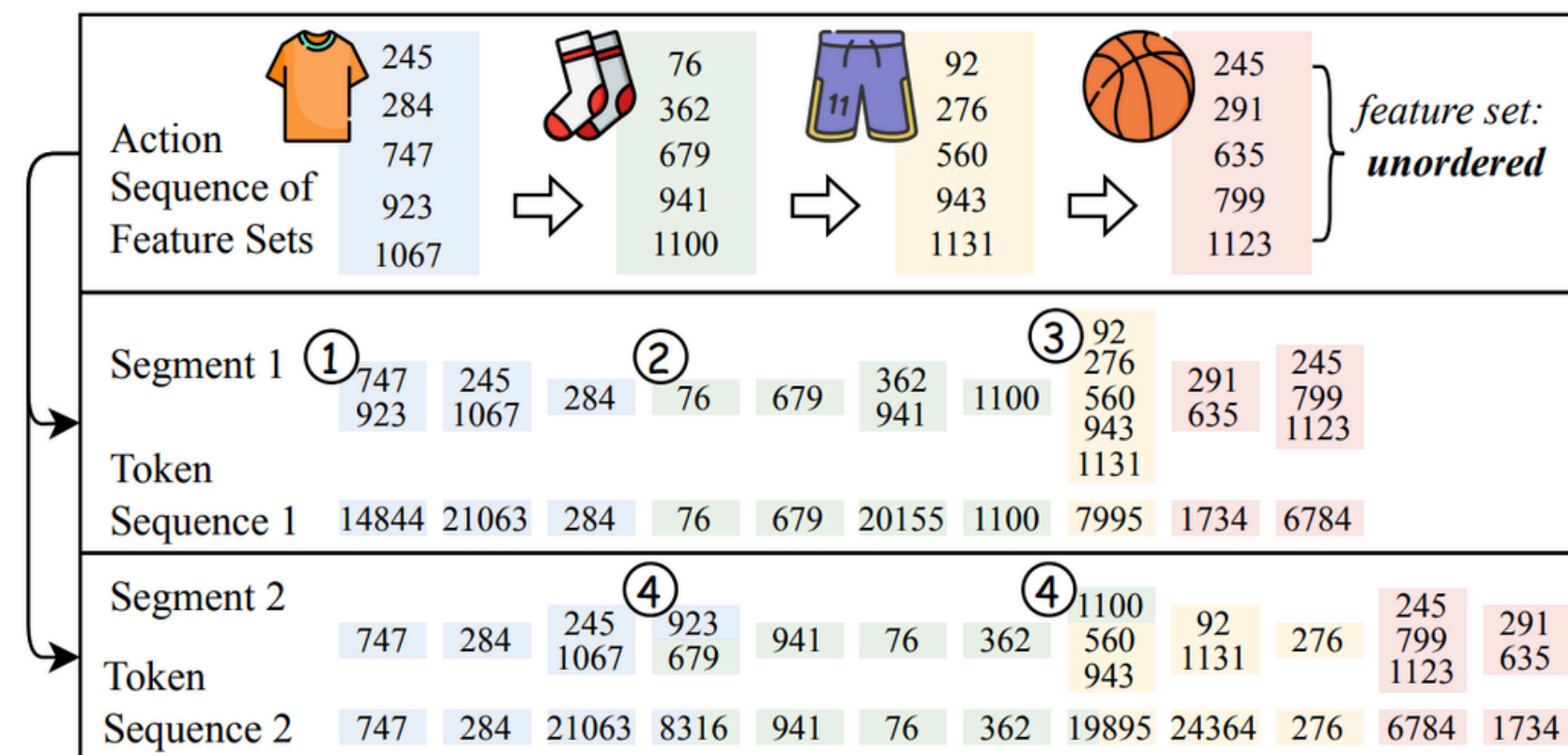


Figure 1. Ilustración del proceso de tokenización de Action Piece.

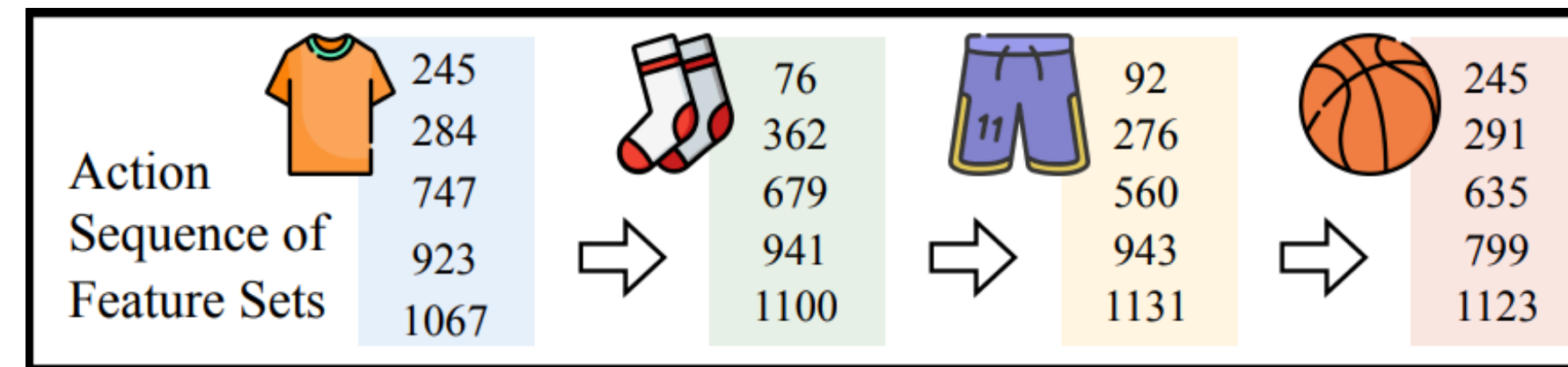
Ejemplo: comprar un short gris puede tener una intención distinta si el contexto previo fue comprar calcetines deportivos versus si fue comprar un pijama

QUÉ SE PREDICE Y CÓMO SE REPRESENTA LA ACCIÓN

- **Definición:** Dado el **historial de acciones** de un usuario organizado cronológicamente $S = \{i_1, i_2, \dots, i_t\}$
- **Acción** = conjunto desordenado de features:
ítem $i_j \rightarrow A_j$; $m = |A_j|$ features.
 - ➔ **Ejemplo:** "El usuario compró/reseñó el calcetín ayer". La secuencia de acciones es el historial cronológico de todo lo que el usuario ha hecho
- **Objetivo:** Predecir el siguiente ítem i_{t+1} con el que interactuará.

$i_t \rightarrow ?$

Secuencia de conjuntos
orden temporal entre conjuntos →



Orden ENTRE conjuntos pero, sin orden DENTRO del conjunto

Permite features generales (categoría, marca, precio) sin imponer un orden artificial, a diferencia de los semantic IDs ordenados.

LAS 3 GRANDES FAMILIAS:

1

ID-based

modelos clásicos, representan ítems solo con un ID aprendido (SASRec, BERT4Rec, etc.)

2

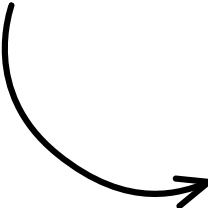
Feature+ID

modelos híbridos, combinan features del ítem (texto, imágenes) con IDs. Más ricos, pero más complejos.

3

Generativos

modelos que generan el identificador del ítem token a token usando un LM (TIGER, SPM-SID, etc.)



Action Piece está aquí

Dos ejes ordenan el campo: ¿el tokenizador usa el contexto? ¿trata las features sin orden?

Tokenización	Ejemplo (baseline)	Contextual	Unordered
Product Quantization	VQ-Rec (Hou et al., 2023)	X	✓
Hierarchical Clustering	P5-CID (Hua et al., 2023)	X	X
Residual Quantization	TIGER (Rajput et al., 2023)	X	X
Text Tokenization	LMIndexer (Jin et al., 2024)	X	X
Raw Features	HSTU (Zhai et al., 2024)	X	X
SentencePiece	SPM-SID (Singh et al., 2024)	X	X
ActionPiece	(este paper)	✓	✓

ActionPiece es el único ✓ en Contextual y el único ✓✓. La brecha no es "falta un método más": nadie tokeniza con contexto.

LOS DOS BASELINES QUE IMPORTAN

TIGER

NeurIPS 2023

- Semantic ID = tupla ordenada de códigos (grueso→fino) vía RQ-VAE.
- Un T5 los predice de forma autoregresiva.
- **Es el backbone de ActionPiece → aísla el efecto del tokenizador.**

SPM-SID

RecSys 2024 · mejor baseline

- SentencePiece sobre semantic IDs: fusiona patrones frecuentes.
- ⚠ Originalmente para ranking; aquí adaptado a rec. generativa.
- **Vocabulario más grande → el rival más fuerte a vencer.**



Ambos suben de nivel el vocabulario como BPE, pero solo ActionPiece mira el contexto entre acciones.

Contribución

Primer tokenizador de acciones consciente del contexto. Tres piezas:

1

Vocabulario

co-ocurrencia ponderada intra/inter-conjunto

2

Implementación eficiente

listas doblemente enlazadas + nodos intermedios

3

Set Permutation Regularization (SPR)

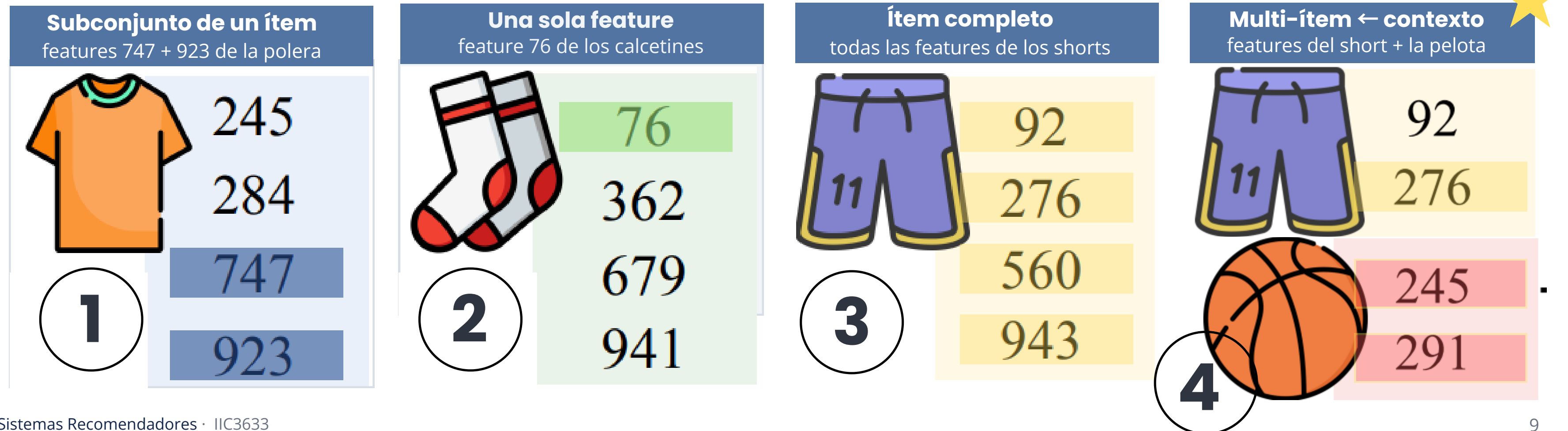
aumenta datos en entrenamiento y hace ensembling en inferencia.

DOS FASES: CONSTRUIR VOCABULARIO Y SEGMENTAR



Flujo de la solución

Cuatro tipos de token (ejemplo de la Figura 1)



“BPE PARA CONJUNTOS”: ITERAR HASTA $|V| = Q$

1 Count

- Contar co-ocurrencias ponderadas en todo el corpus.
- Elegir el par con mayor concurrencia (c_u, c_v).

↺ iterar

2 Update

- Fusionar el par en un nuevo token c_{new} .
- Guardar la regla $(c_u, c_v) \rightarrow c_{\text{new}}$.



Dos tipos de co-ocurrencia: dentro del conjunto (intra) y entre conjuntos adyacentes (inter ← el contexto).

POR QUÉ NO USAR FRECUENCIA CRUDA

- Los conjuntos son desordenados.
- Se pondera por la probabilidad de que dos tokens queden adyacentes al permutar y aplanar la secuencia.
- **La intuición importa tanto como la fórmula.**

Ec. (1) Mesmo Conjunto

$$P(c_1, c_2) = \frac{|\mathcal{A}_i| - 1}{\binom{|\mathcal{A}_i|}{2}} = \frac{2}{|\mathcal{A}_i|}, \quad (1)$$

Ec. (2) Conjuntos adyacentes ← contexto

$$P(c_1, c_3) = \frac{1}{|\mathcal{A}_i| \times |\mathcal{A}_{i+1}|}, \quad \begin{array}{l} c_1 \in \mathcal{A}_i, \\ c_3 \in \mathcal{A}_{i+1}. \end{array} \quad (2)$$

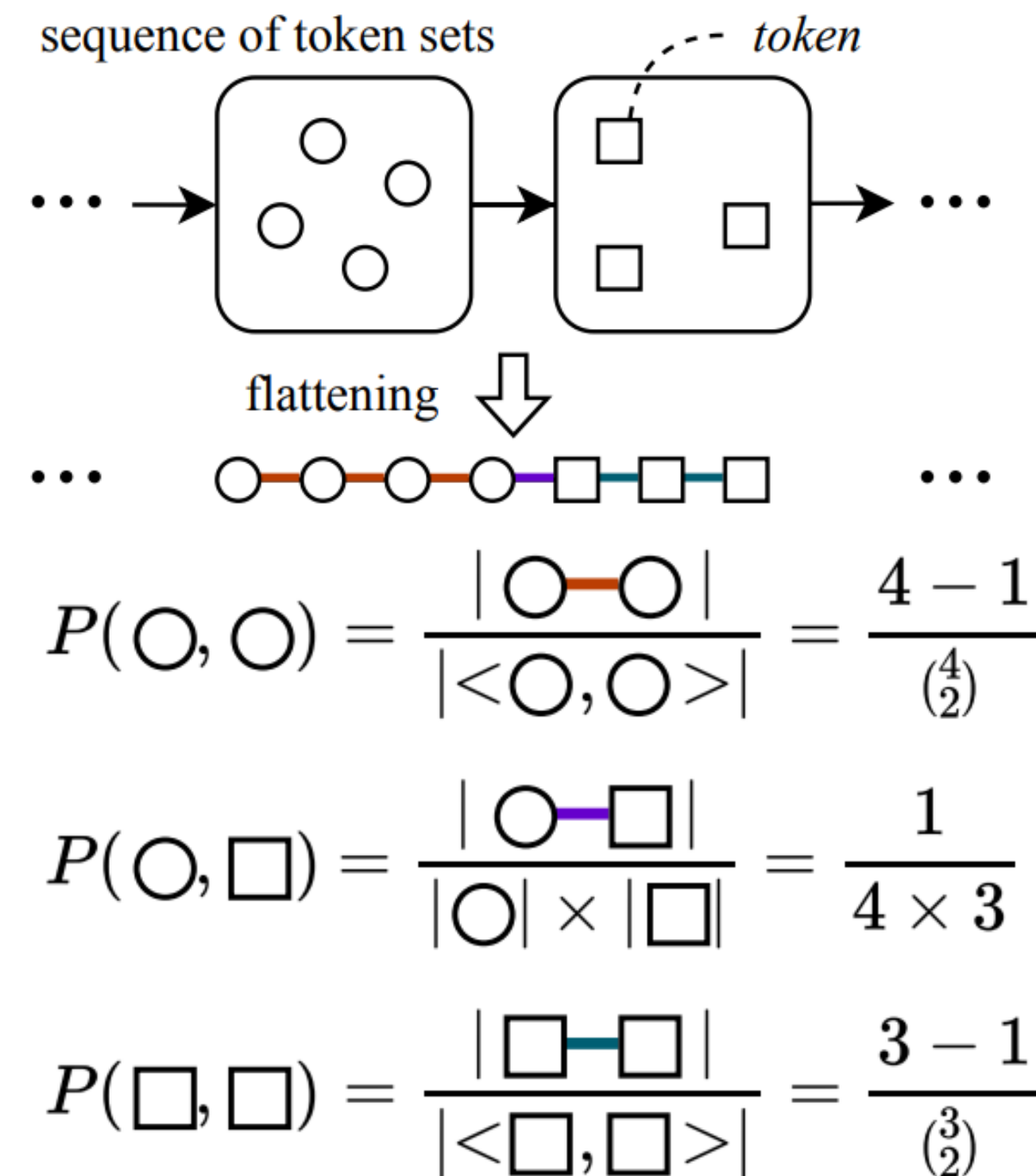


Figure 2. Ilustración de cómo se cuentan los pesos de los pares de tokensco-ocurrentes durante la construcción del vocabulario.

HACER VIABLE LA CONSTRUCCIÓN DEL VOCABULARIO

- **Reto:** Fusionar tokens entre conjuntos aumenta el tamaño de los conjuntos.
- **Nodo intermedio:** entre dos “nodos de acción”, guarda el token que combina features de ambas acciones.
- **Estructuras:** listas doblemente enlazadas + índices invertidos + max-heap.
- **Heap con lazy update:** no se recalcula todo en cada iteración.

Complejidad original:

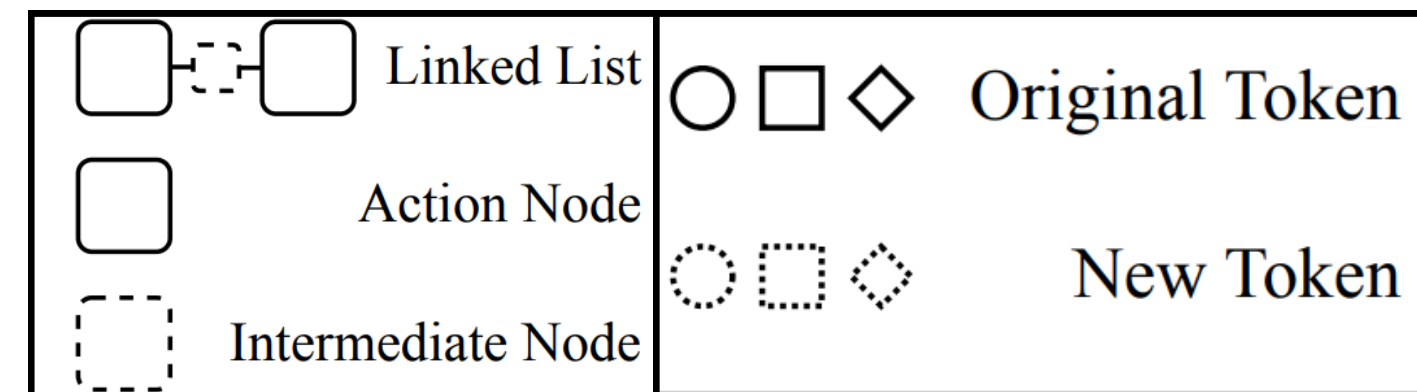
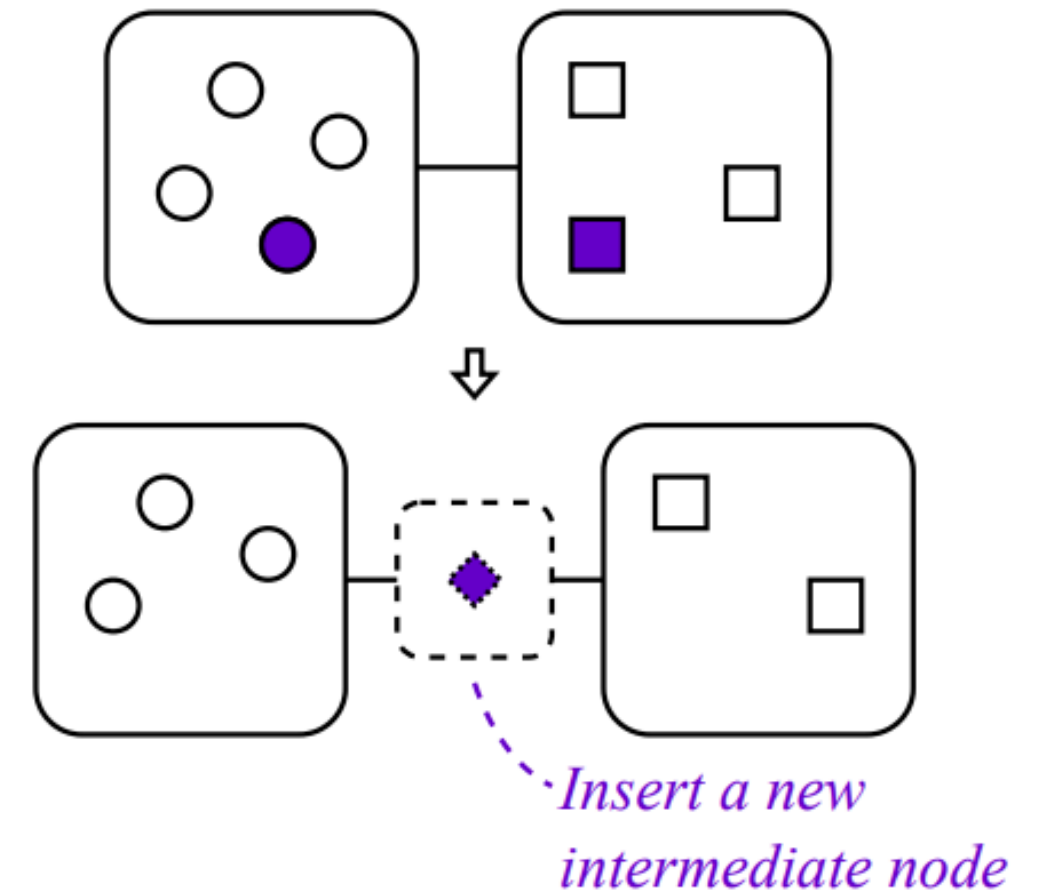
$$O(QNLm^2)$$



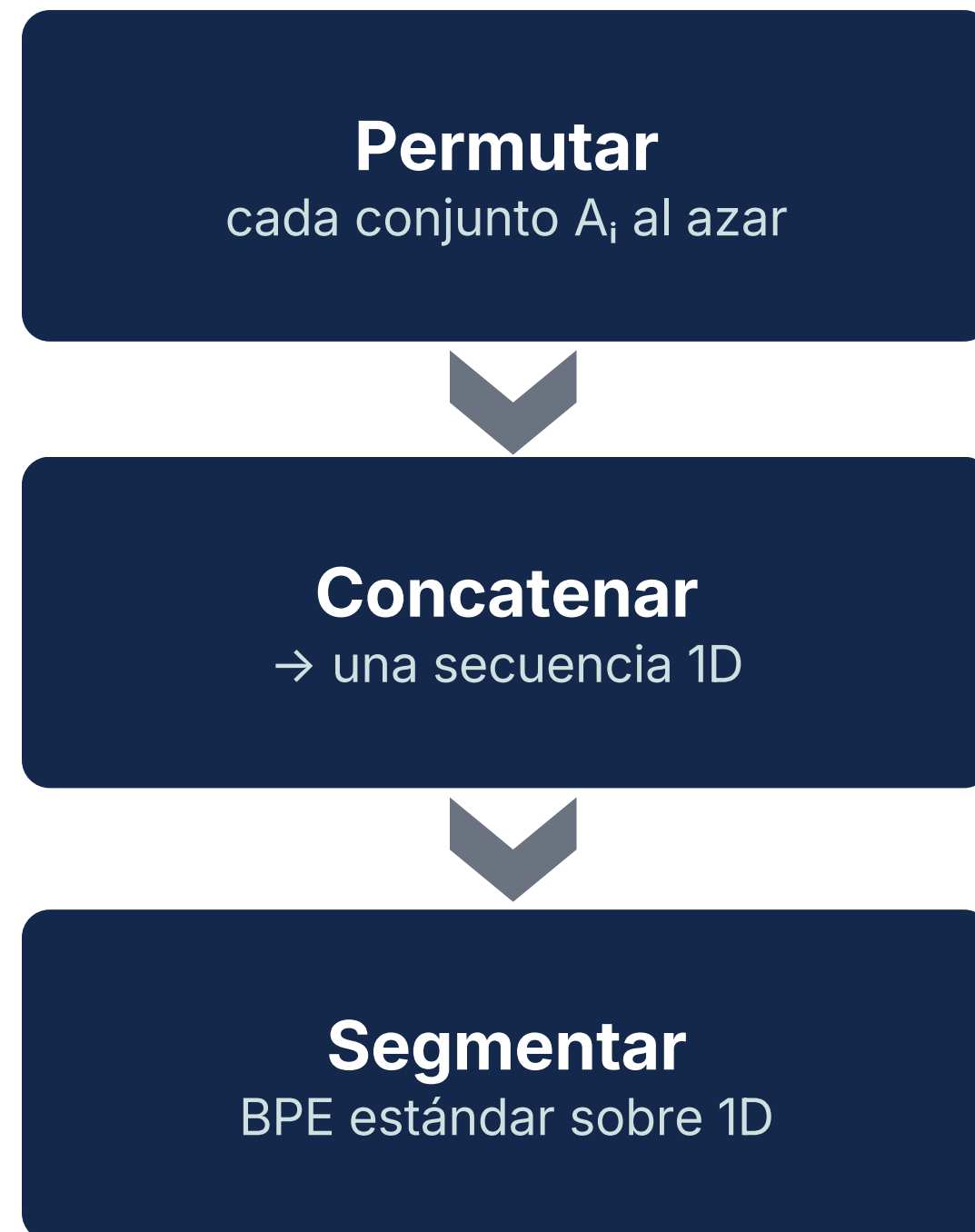
Aplicando nodo intermedio:

$$O(\log Q \times \log H \times NLm^2)$$

Merge tokens in
two adjacent action nodes



¿Cómo Funciona?



Algorithm 4 Segmentation via Set Permutation Regularization (SPR) (Section 3.2.2)

input Action sequence S , merge rules $\mathcal{R}$

output Segmented token sequences C

```
1:  $C \leftarrow []$  # Initialize permuted initial token sequence
2: for all token set  $\mathcal{A}_i \in S$  do
3:   Generate random permutation of  $\mathcal{A}_i$  as  $[c_1, c_2, \dots, c_{|\mathcal{A}_i|}]$ 
4:   Extend  $C$  with  $[c_1, c_2, \dots, c_{|\mathcal{A}_i|}]$  # Concatenate permutations
5: end for
6:
7: # Apply BPE (Sennrich et al., 2016) segmentation on permuted sequence
8: repeat
9:    $\mathcal{R}' \leftarrow \emptyset$  # Candidate merge rules
10:  for  $i \leftarrow 0$  to  $|C| - 1$  do
11:    if  $\{(c_i, c_{i+1}) \rightarrow c'\} \in \mathcal{R}$  then
12:       $\mathcal{R}' \leftarrow \mathcal{R}' \cup \{(c_i, c_{i+1}) \rightarrow c'\}$ 
13:    end if
14:  end for
15:  Select  $\{(c_k, c_{k+1}) \rightarrow c'\} \in \mathcal{R}'$  with the smallest index among all merge rules  $\mathcal{R}$ 
16:   $C \leftarrow [c_1, \dots, c_{k-1}, c', c_{k+2}, \dots]$  # Replace  $(c_k, c_{k+1})$  with a new token  $c'$ 
17: until  $\mathcal{R}'$  is  $\emptyset$ 
return  $C$ 
```

Distintas permutaciones → distintas segmentaciones con la misma semántica.

Datasets — Amazon Reviews (muy sparse, $\sim 10^{-4}$)

Dataset	#Users	#Items	#Actions	Avg. t	Sparsity
Sports	35 598	18 357	260 739	8.32	4.0e-4
Beauty	22 363	12 101	176 139	8.87	6.5e-4
CDs	75 258	64 443	1 022 334	14.58	2.1e-4

Cada review = una acción, ordenadas cronológicamente. CDs es el grande, para escalabilidad.

Métricas (K = 5, 10)

Recall@K: ¿el ítem correcto está en el top-K? (cobertura)

NDCG@K: además premia su posición (calidad del ranking)

Protocolo

Leave-last-out: último ítem = test, penúltimo = validación.

5 semillas (2024–2028) → promedio $\pm$ desviación. Truncado: 20 acciones (generativos) / 50 (resto).

★ **ActionPiece > todos los generativos: el contexto es el diferencial.**

Mejora NDCG@10

Sports +12.82% · Beauty +6.00% · CDs +6.37%

Datasets	Metric	ID-based		Feature + ID			Generative						Improv.
		BERT4Rec	SASRec	FDSA	S ³ -Rec	VQ-Rec	P5-CID	TIGER	LMIndexer	HSTU	SPM-SID	ActionPiece	
Sports	R@5	0.0115	0.0233	0.0182	0.0251	0.0181	0.0287	0.0264	0.0222	0.0258	<u>0.0280</u>	0.0316 ± 0.0005	+12.86%
	N@5	0.0075	0.0154	0.0122	0.0161	0.0132	0.0179	<u>0.0181</u>	0.0142	0.0165	<u>0.0180</u>	0.0205 ± 0.0002	+11.71%
	R@10	0.0191	0.0350	0.0288	0.0385	0.0251	0.0426	<u>0.0400</u>	—	0.0414	<u>0.0446</u>	0.0500 ± 0.0007	+12.11%
	N@10	0.0099	0.0192	0.0156	0.0204	0.0154	0.0224	0.0225	—	0.0215	<u>0.0234</u>	0.0264 ± 0.0003	+12.82%
Beauty	R@5	0.0203	0.0387	0.0267	0.0387	0.0434	0.0468	0.0454	0.0415	0.0469	<u>0.0475</u>	0.0511 ± 0.0014	+7.58%
	N@5	0.0124	0.0249	0.0163	0.0244	0.0311	0.0315	0.0321	0.0262	0.0314	<u>0.0321</u>	0.0340 ± 0.0011	+5.92%
	R@10	0.0347	0.0605	0.0407	0.0647	<u>0.0741</u>	0.0701	0.0648	—	0.0704	0.0714	0.0775 ± 0.0017	+4.59%
	N@10	0.0170	0.0318	0.0208	0.0327	0.0372	<u>0.0400</u>	0.0384	—	0.0389	0.0399	0.0424 ± 0.0011	+6.00%
CDs	R@5	0.0326	0.0351	0.0226	0.0213	0.0314	0.0505	0.0492	—	0.0417	<u>0.0509</u>	0.0544 ± 0.0005	+6.88%
	N@5	0.0201	0.0177	0.0137	0.0130	0.0209	0.0326	0.0329	—	0.0275	<u>0.0337</u>	0.0359 ± 0.0004	+6.53%
	R@10	0.0547	0.0619	0.0378	0.0375	0.0485	<u>0.0785</u>	0.0748	—	0.0638	0.0778	0.0830 ± 0.0008	+5.73%
	N@10	0.0271	0.0263	0.0186	0.0182	0.0264	0.0416	0.0411	—	0.0346	<u>0.0424</u>	0.0451 ± 0.0005	+6.37%

El mejor baseline varía por celda, columna Improv. compara con el más fuerte de cada celda;
SPM-SID es el mejor en promedio, no el mejor de cada celda.

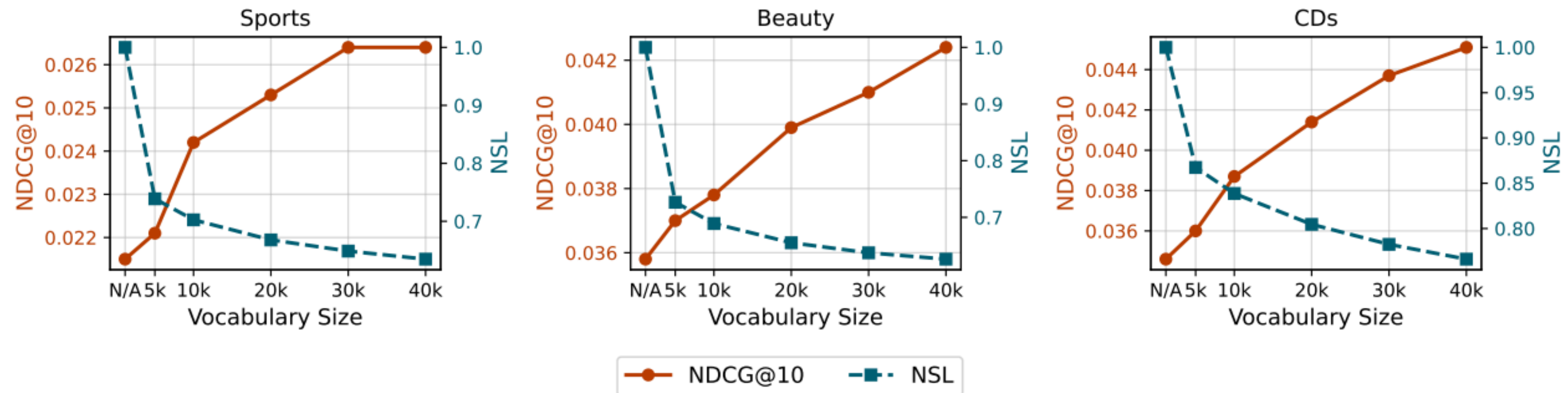


- **Escalar el vocabulario NO basta:** el problema es cómo se escala
- **Quitar piezas baja el rendimiento:** Quitar contexto (2.2) o la ponderación (2.3) baja el rendimiento, ambas piezas importan.
- **Todas son peores que el completo:** SPR aplicado a un tokenizador ciego al contexto (TIGER) no mejora. SPR amplifica una buena tokenización, no la sustituye.

Variants	Sports	Beauty	CDs
<i>TIGER with varying vocabulary sizes</i>			
(1.1) TIGER - 192 (4×48)	0.0231	0.0362	N/A [†]
(1.2) TIGER - 768 (3×2^8)	0.0220	0.0378	0.0331
(1.3) TIGER - 1k (4×2^8)	0.0225	0.0384	0.0411
(1.4) TIGER-49k (6×2^{13})	0.0162	0.0317	0.0338
(1.5) TIGER-66k (4×2^{14})	0.0194	N/A [‡]	0.0319
<i>Vocabulary construction</i>			
(2.1) w/o tokenization	0.0215	0.0389	0.0346
(2.2) w/o context-aware	0.0258	0.0416	0.0429
(2.3) w/o weighted counting	0.0257	0.0412	0.0435
<i>Set permutation regularization</i>			
(3.1) only for inference	0.0192	0.0316	0.0329
(3.2) only for training	0.0244	0.0387	0.0422
(3.3) TIGER + SPR	0.0202	0.0330	0.0351
ActionPiece (40k)	0.0264	0.0424	0.0451



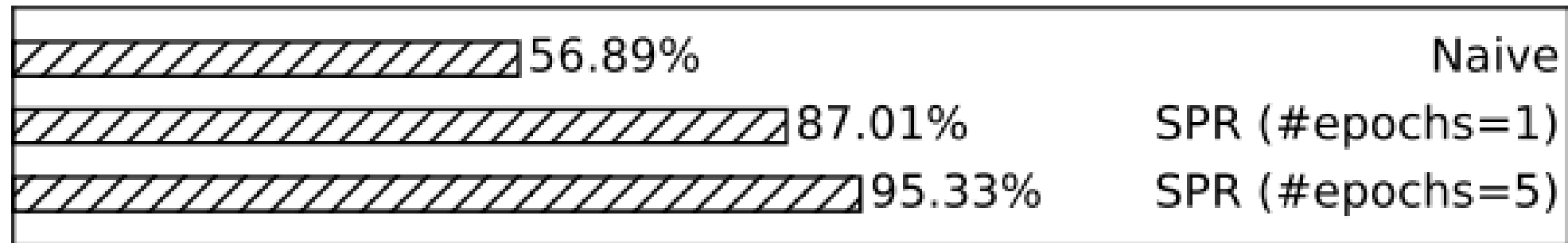
Rendimiento y longitud vs tamaño de vocabulario



**Más vocabulario → mejor NDCG y secuencias más cortas;
menos vocabulario → menos parámetros (memoria).
Es un trade-off rendimiento/memoria/longitud.**



Utilización de tokens



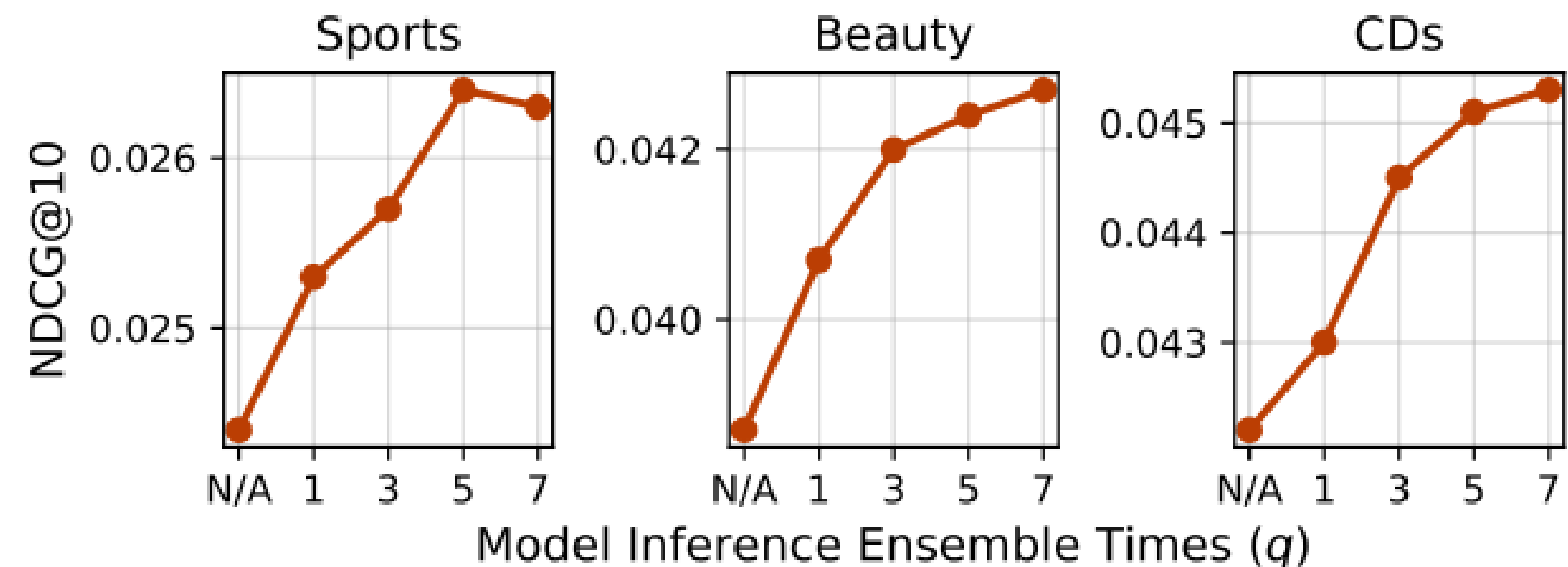
Token Utilization Rate (%)

- SPR hace que casi todo el vocabulario participe.

→ mejor generalización.

- NDCG@10 sube con q , pero satura en $q=5-7$ → rendimientos decrecientes.

- Por eso el paper usa $q=5$ (trade-off rend./costo de inferencia).





Reproducción en Google Colab Pro+

Métrica	Reproducido (1 semilla)	Paper (5 semillas)	Desviación %
recall@5	0.0187	0.0316	-41.0
ndcg@5	0.0119	0.0205	-41.7
recall@10	0.0322	0.0500	-35.7
ndcg@10	0.0163	0.0264	-38.4

1**Diferencias con ejecución del paper:**

Vocabulario (20k vs 40k del paper), Semilla (1 vs 5 del paper), conjunto de validación.

2**Similitudes:**

Con ensemble (q=5), 200 épocas.

```
=== Ablation: ensemble ON (q=5) vs OFF ===  
Métrica      ON (q=5)      OFF  
recall@5     0.0187        0.0171  
ndcg@5       0.0119        0.0108  
recall@10    0.0322        0.0290  
ndcg@10      0.0163        0.0147
```

TRES IDEAS PARA LLEVARSE

1

Contexto

La tokenización con contexto mejora la recomendación.

2

Vocabulario

Conteo ponderado eficiente: un BPE generalizado a conjuntos.

3

SPR

Una misma idea = augmentation + ensembling.

- Hou, Y., Ni, J., He, Z., Sachdeva, N., Kang, W.-C., Chi, E. H., McAuley, J., & Cheng, D. Z. (2025). ActionPiece: Contextually tokenizing action sequences for generative recommendation. Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR 267. <https://doi.org/10.48550/arXiv.2502.13581>
- Rajput, S., Mehta, N., Singh, A., Keshavan, R. H., Vu, T., Heldt, L., Hong, L., Tay, Y., Tran, V. Q., Samost, J., Kula, M., Chi, E. H., & Sathiamoorthy, M. (2023). Recommender systems with generative retrieval. Advances in Neural Information Processing Systems (NeurIPS), 36. <https://doi.org/10.48550/arXiv.2305.05065>
- Singh, A., Vu, T., Mehta, N., Keshavan, R., Sathiamoorthy, M., Zheng, Y., Hong, L., Heldt, L., Wei, L., Tandon, D., Chi, E. H., & Yi, X. (2024). Better generalization with semantic IDs: A case study in ranking for recommendations. Proceedings of the 18th ACM Conference on Recommender Systems (RecSys), 1039–1044. <https://doi.org/10.48550/arXiv.2306.08121>
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL), 1715–1725. <https://doi.org/10.48550/arXiv.1508.07909>
- Lee, D., Kim, C., Kim, S., Cho, M., & Han, W.-S. (2022). Autoregressive image generation using residual quantization. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 11523–11532. <https://doi.org/10.48550/arXiv.2203.01941> (RQ-VAE)
- Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations, 66–71. <https://doi.org/10.48550/arXiv.1808.06226>
- Kang, W.-C., & McAuley, J. (2018). Self-attentive sequential recommendation. IEEE International Conference on Data Mining (ICDM), 197–206. <https://doi.org/10.48550/arXiv.1808.09781> (SASRec)

RECOMENDACIÓN GENERATIVA · TOKENIZACIÓN CONTEXTUAL

ActionPiece

Contextually Tokenizing Action Sequences for
Generative Recommendation

Seminario · Sistemas Recomendadores

Yupeng Hou · Jianmo Ni · Zhankui He · Noveen Sachdeva · Wang-Cheng Kang · Ed H. Chi · Julian McAuley · Derek Zhiyuan Cheng

Presentado por: **Benjamín Suazo y Pablo Lagos**

09 de junio de 2026



ANEXOS

Material de apoyo para preguntas



A1 · Notación

A2 · Conceptos

A3 · Algoritmos

A4 · Competidores

A5 · Métricas

Símbolo	Significado
$S = \{i_1, \dots, i_t\}$	secuencia de acciones (por ID de ítem)
$A_j ; m = A_j $	conjunto de features; nº de features (5 en la implementación)
$f_{j,k} \in F_k$	k-ésima feature del ítem; conjunto de opciones
$C = \{c_1, \dots, c_l\}$	secuencia de tokens; $l > t$ típicamente
$V ; Q = V $	vocabulario y su tamaño objetivo
$(c_u, c_v) \rightarrow c_{new}$	una regla de fusión
$P(c, c')$	prob. de adyacencia al aplanar la secuencia de conjuntos
N , L	nº y longitud media de secuencias en el corpus
$H = O(NLm)$	tamaño máximo del heap
q	nº de segmentaciones de SPR en inferencia

Rec. generativa (GR)

Tokenizar acciones y generar los tokens del siguiente ítem de forma autoregresiva.

SentencePiece / SPM

Implementación de BPE/Unigram sobre texto crudo; la usa SPM-SID.

RQ-VAE / OPQ

Cuantización: RQ es jerárquica (grueso→fino, ordenada); OPQ no impone orden.

BPE

Tokenizador subword: fusiona iterativamente el par adyacente más frecuente.

Semantic ID

Tupla ordenada de códigos que representa un ítem por su contenido (TIGER).

NSL

Longitud media de la secuencia tokenizada (menor = más compacta).



Algorithm 1 ActionPiece Vocabulary Construction

input Sequence corpus $\mathcal{S}'$, initial tokens $\mathcal{V}_0$, target size Q

output Merge rules $\mathcal{R}$, constructed vocabulary $\mathcal{V}$

```
1: Initialize vocabulary  $\mathcal{V} \leftarrow \mathcal{V}_0$  # Each initial token corresponds
   to one unique item feature
2:  $\mathcal{R} \leftarrow \emptyset$ 
3: while  $|\mathcal{V}| < Q$  do
4:   # Count: accumulate weighted token co-occurrences
5:    $\text{count}(\cdot, \cdot) \leftarrow \text{Count}(\mathcal{S}', \mathcal{V})$  # Algorithm 2
6:   # Update: merge a frequent token pair into a new token
7:   Select  $(c_u, c_v) \leftarrow \arg \max_{(c_i, c_j)} \text{count}(c_i, c_j)$ 
8:    $\mathcal{S}' \leftarrow \text{Update}(\mathcal{S}', \{(c_u, c_v) \rightarrow c_{\text{new}}\})$  # Algorithm 3
9:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(c_u, c_v) \rightarrow c_{\text{new}}\}$  # New merge rule
10:   $\mathcal{V} \leftarrow \mathcal{V} \cup \{c_{\text{new}}\}$  # Add new token to the vocabulary
11: end while
return  $\mathcal{R}, \mathcal{V}$ 
```

ID-based

- SASRec: Transformer self-attentivo, BCE.
- BERT4Rec: encoder bidireccional, masked.

Feature + ID

- FDSA: atención sobre features e IDs por separado.
- S^3 -Rec: pre-entrenamiento self-supervised.
- VQ-Rec: PQ sobre embeddings de texto.

Generativos

- P5-CID: clustering jerárquico.
- TIGER: RQ-VAE + T5 (backbone).
- LMIndexer: texto $\rightarrow$ semantic IDs.
- HSTU: features crudas, sin merge (NSL=1).
- SPM-SID: SentencePiece sobre semantic IDs.

Features (todos): título + precio + marca + ... $\rightarrow$ sentence-t5-base (768-dim) $\rightarrow$ cuantización. TIGER usa RQ (3 $\times$ 256 + ID); ActionPiece usa OPQ (4 $\times$ 256 + ID, sin orden).



Hyperparameter	Sports	Beauty	CDs
learning_rate	0.005	0.001	0.001
warmup_steps	10,000	10,000	10,000
dropout_rate	0.1	0.1	0.1
weight_decay	0.15	0.15	0.07
vocabulary_size	40,000	40,000	40,000
n_inference_segments	5	5	5
beam_size	50	50	50
num_layers	4	4	4
d_model	128	128	256
d_ff	1,024	1,024	2,048
num_heads	6	6	6
d_kv	64	64	64
optimizer	adamw	adamw	adamw
lr_scheduler	cosine	cosine	cosine
train_batch_size	256	256	256
max_epochs	200	200	200
early_stop_patience	20	20	20